ML Toolbox

Credits

- Harvard Institute for Applied Computational Science - IACS

Virtual Envirornments

Problem Statement: Solving Dependency Hell

Why should we use virtual environment?

- Virtual environments help to make development and use of code more streamlined.
- Virtual environments keep dependencies in separate “sandboxes” so you can switch between both applications easily and get them running.
- Given an operating system and hardware, we can get the exact code environment set up using different technologies. This is key to understand the trade off among the different technologies presented in this class.

What might be the problem here?

What are virtual environments then?

A virtual environment is a directory with the following components:

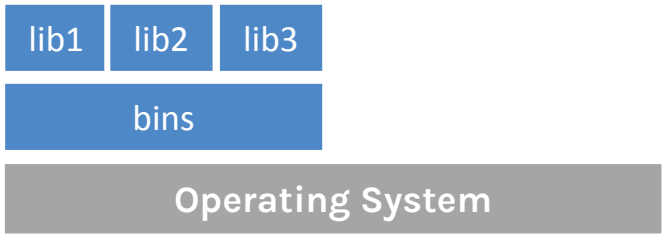
- site_packages/ directory where third-party libraries are installed
- links [really symlinks] to the executables on your system
- some scripts that ensure that the code uses the interpreter and site packages in the virtual environment

> Adapted from CS207 <

So basically when you "activate" an environment, they temporarily modify your system's PATH variable so that when you type

Why should we use virtual environment?

- Maggie took cs109a, she used to run her Jupyter notebooks from anaconda prompt. Every time she installed a module it was placed in the either of `bin, lib, share, include` folders and she could import it in and used it without any issue.




```
$ which python
/c/Users/maggie/Anaconda3/python
```

Maggie

Why should we use virtual environment?

- Maggie took cs109a, she used to run her Jupyter notebooks from anaconda prompt. Every time she installed a module it was placed in the either of `bin, lib, share, include` folders and she could import it in and used it without any issue.

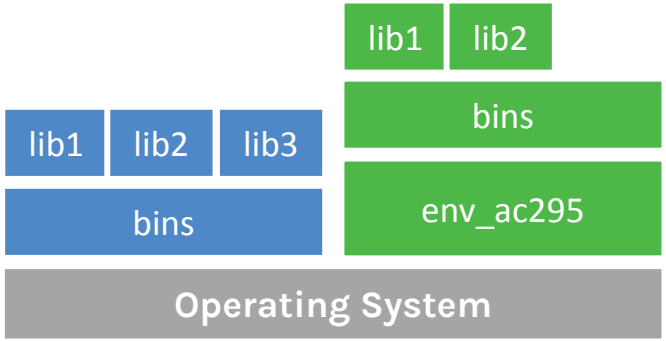
Maggie

```
$ which python
/c/Users/maggie/Anaconda3/python
```

This is problematic

Why should we use virtual environment?

- Maggie starts taking ac295 and she thinks that would be good to isolate the new environment from the previous enviroments avoiding any conflict with the installed packages. She adds a layer of abstraction called virtual environment that helps her keep the modules organized and avoid misbehaviors while developing a new project.

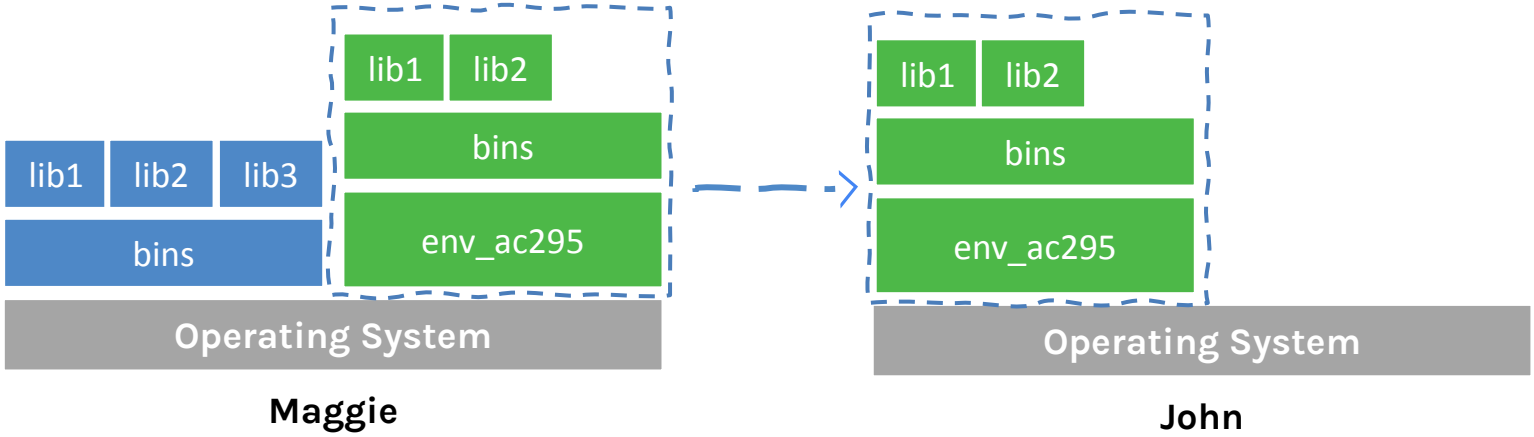


Maggie

```
$ which python
/c/Users/maggie/Anaconda3/envs/env_ac295/python
```

Why should we use virtual environment?

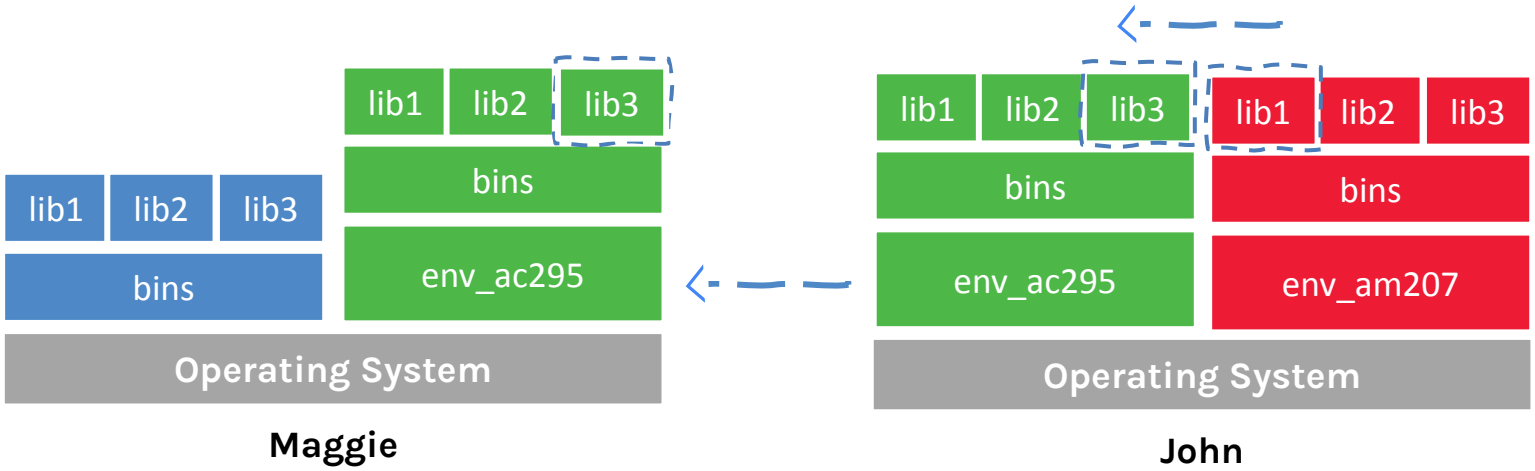
- Maggie collaborates with John for the final project and shares with him the environment she is working on through .yml file.



.yml -> requirements.txt (a precise list of every package and its exact version needed for the project)

Why should we use virtual environment?

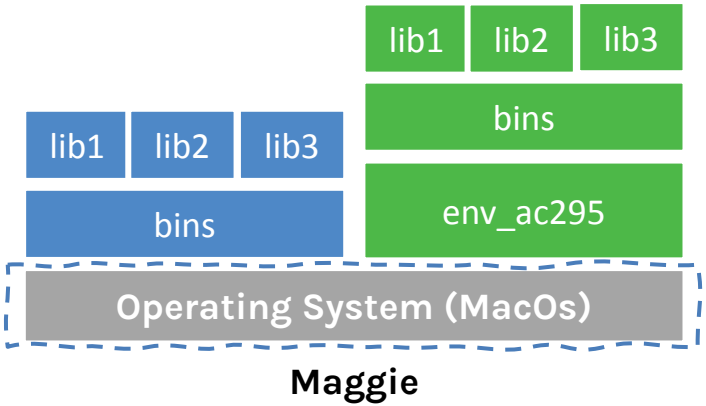
- John experiments a new method he learned in another class and adds a new library to the working environment. After seeing a tremendous improvements he sends Maggie back his code and a new .yml file. She can now update her environment and replicate the experiment.



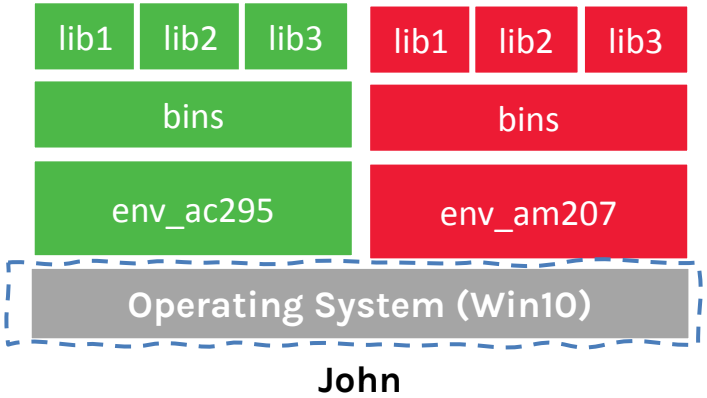
This should work especially if you are using VCS (aka Git) workflows

Why should we use virtual environment?

- What could go wrong? Unfortunately, Maggie and John reproduce different results and they think the issue relates to their operating systems. Indeed while Maggie has a MacOS, John uses a Win10.



Maggie



John

Virtual environments

Pros

- Reproducible research
- Explicit dependencies
- Improved engineering collaboration
- Broader skill set

Cons

- Difficulty setting up your environment
- Not isolation
- Does not work across different OS

Virtual environments: virtualenv vs conda

virtualenv

- virtual environments manager embedded in Python
- incorporated into broader tools such as `pipenv`
- allow to install modules using pip package manager

how to use **virtualenv**

- create an environment within your project folder `virtualenv your_env_name`
- it will add a folder called environment_name in your project directory
- activate environment: `source env/bin/activate`
- install requirements using: `pip install package_name=version`
- deactivate environment once done: `deactivate`

Virtual environments in practice (virtualenv vs conda)

conda environment

- virtual environments manager embedded in Anaconda
- allow to use both conda and pip to manage and install packages

how to use **conda**

- create an environment `conda create --name your_env_name python=3.7`
- it will add a folder located within your anaconda installation `/Users/your_username/anaconda3/envs/your_env_name`
- activate environment `conda activate your_env_name` (should appear in your shell)
- install requirements using `conda install package_name=version`
- deactivate environment once done `conda deactivate`
- duplicate your environment using YAML file `conda env export > my_environment.yml`
- to recreate the environment now use `conda env create -f environment.yml`

More on Virtual environments

Further readings

- For detailed discussions on similarities and differences among virtualenv and conda https://jakevdp.github.io/blog/2016/08/25/conda-myths-and-misconceptions/
- More on venv and conda environments
  https://towardsdatascience.com/virtual-environments-104c62d48c54
  https://towardsdatascience.com/getting-started-with-python-environments-using-conda-32e9f2779307

Why should we use virtual machines?

Motivation

- we have our isolated systems and after we set up a similar environment into our colleagues' machines we should get similar results, right? Unfortunately, it is not always the case. Why? Most likely because we run it on **different operating system**.
- even though by using virtual environments we are isolating our computations, we might need to use the same operating system which requires to run "like if" we are in a different machines.
- How can we run the same experiment? Virtual Machines!
- **Isolation!**

So basically:

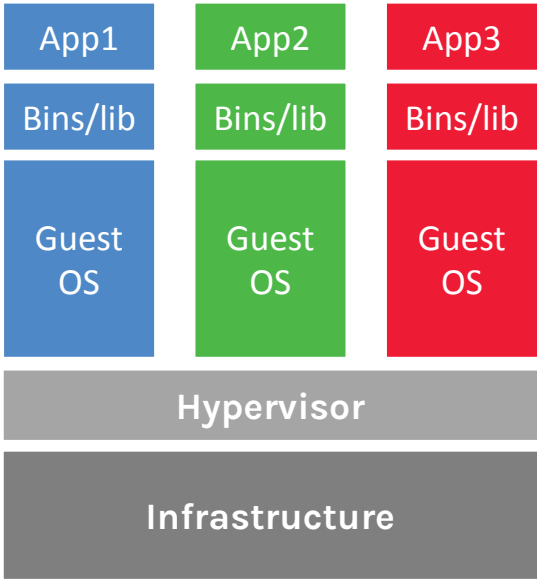
Why should we use virtual machines?(cont)

Advantages

- full autonomy: it works like a separate computer system, it is like run a computer within a computer.
- **very secure**: the software inside the virtual machines can't affect the actual computer.
- lower costs: buy one machine and run multiple operating systems.

What are virtual machines?

- virtual machines have their own virtual hardware: CPUs, memory, hard drives, etc.
- you need a hypervisor that manages different virtual machines on server
- hypervisor can run as many virtual machines as you wish
- operating system is called the "host" while those running in a virtual machine are called "guest“
- You can install a completely different operating system on this virtual machine



Machine Virtualization

https://towardsdatascience.com/how-to-install-a-free-windows-virtual-machine-on-your-mac-bf7cbc05888

Limitations

- Uses hardware in your local machine
- There is overhead associated with virtual machines
    1. guest is not as fast as the host system
    2. Takes long time to start up
    3. may not have the same graphics capabilities

Containers

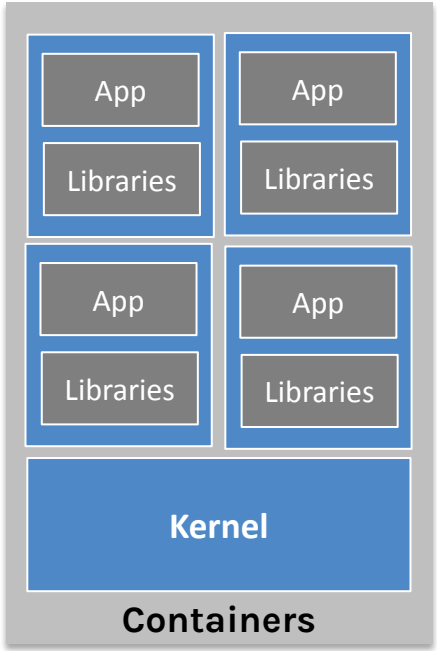
Aka The Best of Both Worlds

Why should we use containers?

- It has the best of the two worlds because it allows:
  1. to create isolate environment using the preferred operating system
  2. to run different operating system without sharing hardware

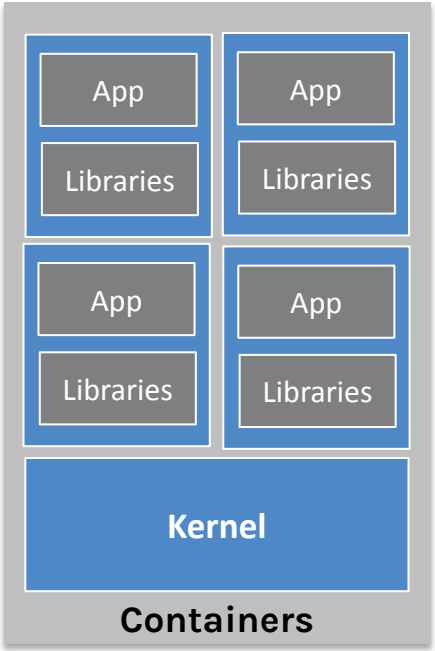
- The advantage of using containers is that they only **virtualize the operating system** and do not require dedicated piece of hardware because they share the same kernel of the hosting system.

- Containers give the impression of a separate operating system however, since they're sharing the kernel, they are much cheaper than a virtual machine.



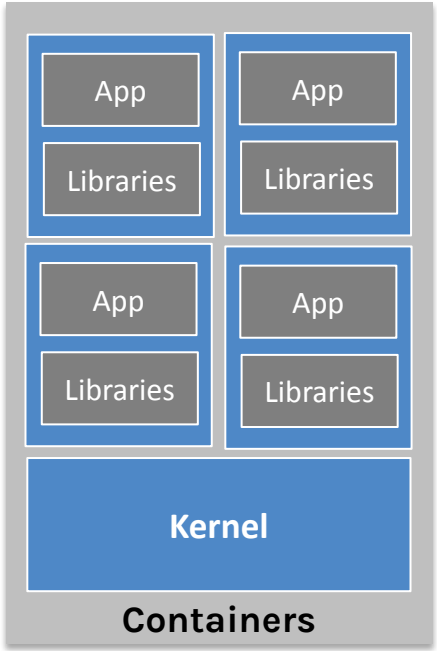
Why should we use containers? (cont)

- With container images, we confine the application code, its runtime, and all its dependencies in a pre-defined format.
- With the same image, you can reproduce as many containers as you wish. Think about the image as the recipe, and the container as the cake ;-) you can make as many cakes as you'd like with a given recipe.
- A container orchestrator is a single controller/management unit that connects multiple nodes together.
- You can create a container on a Window but install an image of a Linux OS inside that container. The container still works on the Window



Why should we use containers? (cont)

- Containers are application-centric methods to deliver high-performing, scalable applications on any infrastructure of your choice
- Containers are best suited to deliver **microservices** by providing portable, isolated virtual environments for applications to run without interference from other running applications
- Containers run container images, it bundles the application along with its runtime and dependencies
- Because they're so lightweight, you can have many containers running at once on your system.



Problem Statement: peak demand

Load balancing

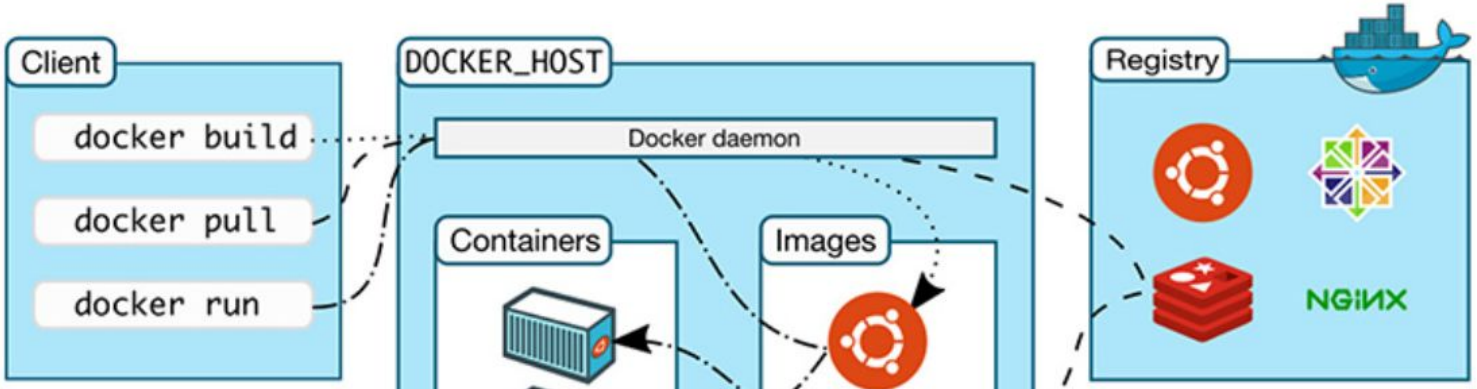
Horizontal vs Vertical Scaling

This is called container orchestration aka Kubernetes

Why containers recap?

- Their startup time is on the order of seconds (vs. minutes for Virtual Machines).
- They provide pseudo-isolation. This means they're still pretty secure, but not as secure at a Virtual Machines.
- A container is deployed from the container image offering an isolated executable environment for the application.
- Containers can be deployed from a specific image on many platforms, such as workstations, Virtual Machines, public cloud, etc.
- Containers are extremely popular, and their popularity is growing.
- One of the first widely used containers was provided by Docker.
- Docker containers can be used to run websites and web applications.
- Multiple containers can be managed by a service called Kubernetes (see next lecture)

The Docker Ecosystem



The **client** allows to run various Docker commands (installed in any OS).

The **docker host** is the server running the Docker daemon.

The **registry** is place to find and download Docker images.

https://nickjanetakis.com/blog/understanding-how-the-docker-daemon-and-docker-cli-work-together

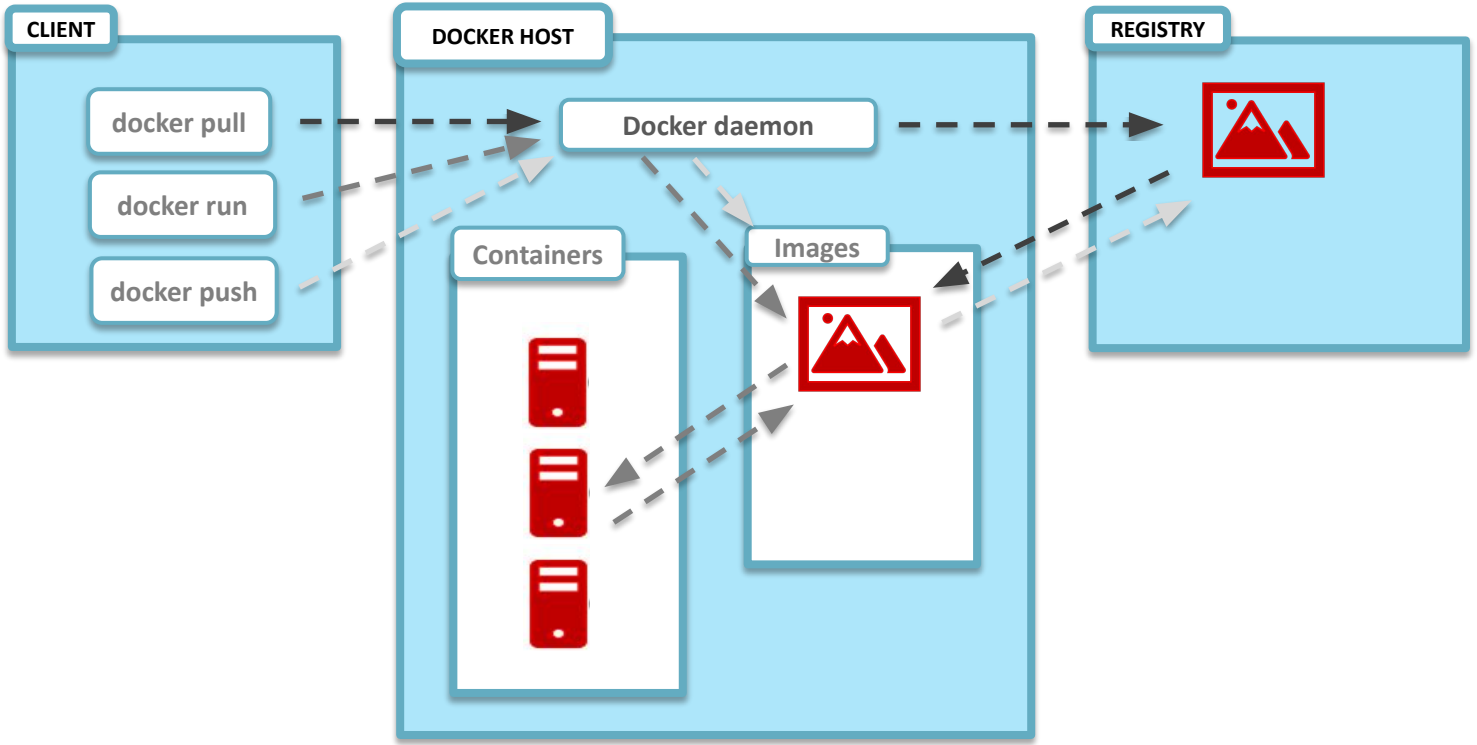
Hands on Containers

Exercise set up

Exercise 1: Pull, modify, and push a Docker image from DockerHub

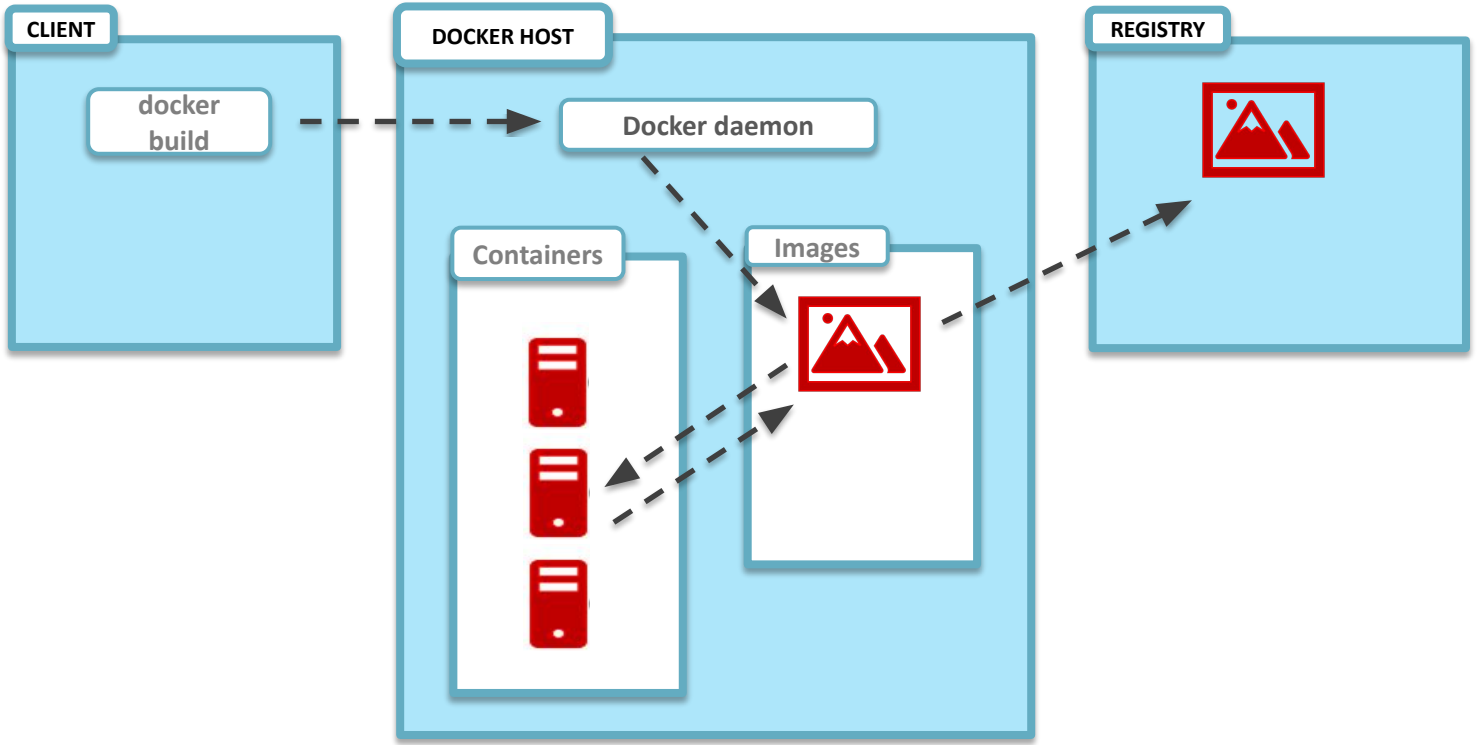
Exercise 2: Build a Docker Image and push it to DockerHub

Exercise 1: pull, modify, and push an image



https://nickjanetakis.com/blog/understanding-how-the-docker-daemon-and-docker-cli-work-together

Exercise 2: build a Docker Image



https://nickjanetakis.com/blog/understanding-how-the-docker-daemon-and-docker-cli-work-together

Lab

DOCKERFILE

```
FROM centos:8

RUN dnf install -y python38
```

FROM centos:8 (base image)

RUN dnf install -y python38

FROM centos:8

RUN dnf install -y python38

Key concept: Work in layers

```
$ docker build .
[+] Building 11.2s (6/6) FINISHED
 => => transferring context: 2B
 => => transferring dockerfile: 83B
 => CACHED [1/2] FROM docker.io/library/centos:8
 => [2/2] RUN dnf install -y python38
 => exporting to image
 => => exporting layers
 => => writing
image sha256:3ca470de8dbd5cb865da679ff805a3bf17de9b34ac6a7236db
```

```
$ docker images
docker images
REPOSITORY    TAG       IMAGE ID            CREATED
<none>        <none>    3ca470de8dbd        15 minutes ago
```

```
$ docker build -t localbuild:removeme .
[+] Building 0.3s (6/6) FINISHED
[...]
 => => writing
image sha256:4c5d79f448647e0ff234923d8f542eea6938e0199440dfc75b
 => => naming to docker.io/library/localbuild:removeme
```

```
$ docker images localbuild
REPOSITORY          TAG                 IMAGE ID            CREATED
localbuild          removeme            3ca470de8dbd        22 minutes
```

```
$ docker tag localbuild:removeme alfredodeza/removeme
$ docker push alfredodeza/removeme
The push refers to repository [docker.io/alfredodeza/removeme]
958488a3c11e: Pushed
291f6e44771a: Pushed
latest: digest: sha256:a022eea71ca955cafb4d38b12c28b9de59dbb3d9
```

Explore > alfredodeza/removeme

alfredodeza/removeme ☆

By alfredodeza • Updated a few seconds ago

Container

```
$ docker run -ti -d --name centos-test --rm centos:8 /bin/bash
1bb9cc3112ed66151166351724989 8bfc9524fc02dedc3ce40b5c4cb982d7bc
```

```
$ docker ps
CONTAINER ID        IMAGE               COMMAND             NAM
1bb9cc3112ed        centos:8            "/bin/bash"         cen
```

```
FROM centos:8

RUN dnf install -y python38

ENTRYPOINT ["/bin/bash"]
```

Docker trivia: Why && ?

```
RUN apk add --no-cache python3&& python3 -m ensurepip && pip3 install pytest
```

```
FROM python:3.7

ARG VERSION

LABEL org.label-schema.version=$VERSION

COPY ./requirements.txt /webapp/requirements.txt

WORKDIR /webapp

RUN pip install -r requirements.txt

COPY webapp/* /webapp

ENTRYPOINT [ "python" ]

CMD [ "app.py" ]
```